

SEAR: Scaling Experiences in Multi-user Augmented Reality

Wenxiao Zhang, Bo Han, *Senior Member, IEEE*, and Pan Hui, *Fellow, IEEE*

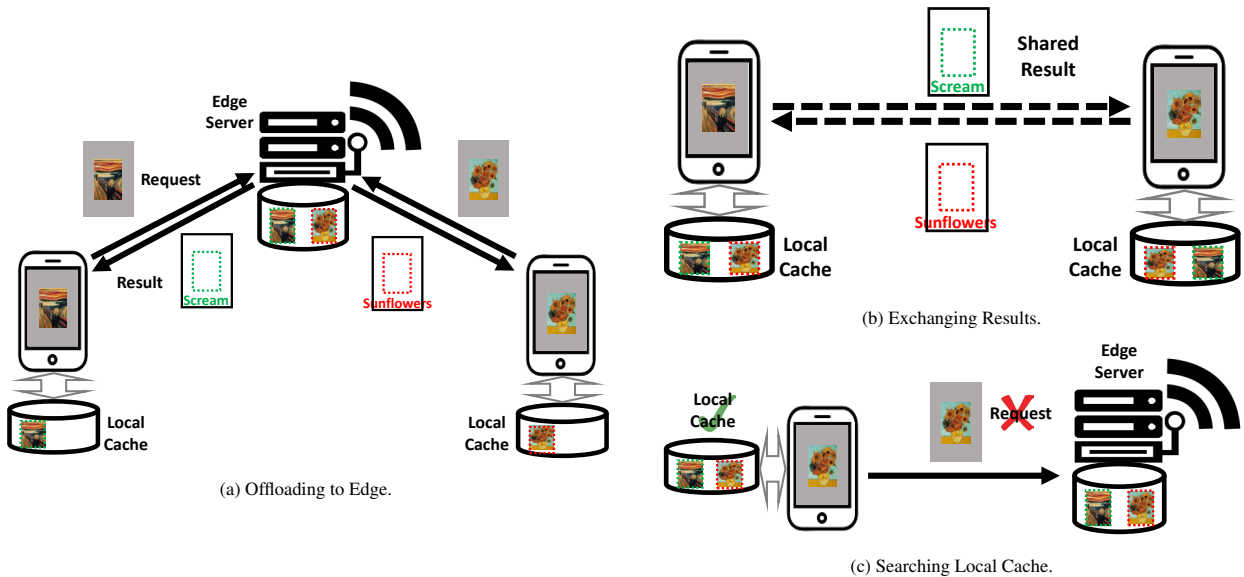


Fig. 1: An example to demonstrate the collaborative local caching framework of SEAR. (a) Under normal conditions, users offload computation-intensive AR tasks to the edge server for improved user experience, in terms of reduced end-to-end latency. (b) Users exchanged the results of offloaded AR tasks and save them in a local cache. (c) When the workload of the edge server is high, users resort to the local cache, instead of waiting for the response from the edge.

Abstract—In this paper, we present the design, implementation, and evaluation of SEAR, a collaborative framework for Scaling Experiences in multi-user Augmented Reality (AR). Most AR systems benefit from computer vision (CV) algorithms to detect, classify, or recognize physical objects for augmentation. A widely used acceleration method for mobile AR is to offload the compute-intensive tasks (e.g., CV algorithms) to the network edge. However, we show that the end-to-end latency, an important metric of mobile AR, may dramatically increase when offloading AR tasks from a large number of concurrent users to the edge. SEAR tackles this scalability issue through the innovation of a lightweight collaborative local caching scheme. Our key observation is that nearby AR users may share some common interests, and may even have overlapped views to augment (e.g., when playing a multi-user AR game). Thus, SEAR opportunistically exchanges the results of offloaded AR tasks among users when feasible and leverages compute resources on mobile devices to relieve, if necessary, the edge workload by intelligently reusing these results. We build a prototype of SEAR to demonstrate its efficacy in scaling AR experiences. We conduct extensive evaluations through both real-world experiments and trace-driven simulations. We observe that SEAR not only reduces the end-to-end latency, by up to $130\times$, compared to the state-of-the-art adaptive edge offloading scheme, but also achieves high object-recognition accuracy for mobile AR.

Index Terms—Augmented Reality (AR), Multi-user AR, Edge-assisted AR, Scalability, User Experience, Latency

1 INTRODUCTION

Augmented Reality (AR) enhances the physical world by augmenting one's perception of reality with overlaid visual, auditory, and haptic sensory information [72]. To identify objects for augmentation, AR systems typically benefit from computer vision (CV) algorithms such as object classification [7, 38] and recognition [27, 28, 70], which are in general computationally intensive and may result in noticeable latency

when executed on mobile devices. Hence, offloading these computation-heavy tasks to the cloud or network edge, especially by leveraging graphics processing unit (GPU) acceleration, is a promising solution to reduce end-to-end latency for mobile AR [38, 39].

Due to the tremendous advantages of mobile edge computing, it will be utilized by multiple applications such as AR, connected cars, and video analytics [6] that can attract thousands of concurrent users. However, given its large footprint for being close to end-users, the edge is resource-constrained regarding both compute and network resources for economic considerations, compared to the centralized cloud infrastructure [43, 67]. This will cause the *scalability issue*, which is overlooked in state-of-the-art offloading solutions for AR [7, 14, 38, 39]. Existing work also unveils that continuously offloading CV tasks to the cloud at 10 hours usage per day would cost roughly \$1,500-3,000 per year and 2.1 TB data per month [22].

In this paper, we propose SEAR (Scaling Experiences in multi-user AR), which is, to the best of our knowledge, the first collaborative framework for addressing the aforementioned scalability issue of edge-assisted AR systems. The design of SEAR is inspired by our prior work

- Wenxiao Zhang was with Hong Kong University of Science and Technology when most of this work was done. E-mail: wzhangal@cse.ust.hk.
- Bo Han is with George Mason University. E-mail: bohan@gmu.edu.
- Pan Hui is with Hong Kong University of Science and Technology. E-mail: panhui@cse.ust.hk.

CARS (collaborative augmented reality for socialization) [71], whose key insight is that AR ties tightly with the physical world. Users of an AR application are likely to be in the vicinity of each other (*e.g.*, during sightseeing [18], classroom education [51], and military training [40]), especially for multi-user AR [4, 8, 20, 66]. While CARS facilitates the interaction among users for collaborative efforts [39, 71], it may deteriorate the aforementioned scalability issues when a large number of users offload their AR tasks to the edge with restricted resources. Thus, we advocate further utilizing the social nature of human beings for sharing results of compute-intensive AR tasks. By doing this, we can potentially improve the scalability of edge-assisted AR systems and enhance the user-perceived quality of experience (QoE).

We illustrate the high-level idea of SEAR using the following example, as shown in Fig. 1. Suppose Alice and Bob are visiting a gallery. SEAR augments their views with the appropriate information (*i.e.*, recognition results and annotations), which is retrieved from an edge server. After that, when they meet in the hallway they can exchange the results from previous runs and save them in a local cache on their mobile devices. This is feasible via mobile opportunistic communications [21, 37, 44]. Later on, Alice enters another exhibit room to visit the painting that Bob just appreciated. It is the peak hour of the gallery and the edge workload is too high to accommodate offloaded tasks in real time. Instead of waiting for the edge server to handle her request, Alice can take advantage of the cached result from Bob for performing the AR task *locally* with the help of SEAR.

Although the above idea of collaborative AR is straightforward, in practice, there are several challenges for designing SEAR as a holistic system. Usually, there is a trade-off between reducing the latency of AR systems and improving their accuracy of object classification/recognition. With a large number of concurrent users, the processing time on the edge will naturally increase due to competition among offloaded tasks (§3). An intuitive solution for optimizing the processing latency is to use lightweight CV algorithms, which will inevitably affect the accuracy [7, 52]. To optimize the computation latency and make SEAR practical, we have to choose a lightweight but less accurate object recognition algorithm for executing on mobile devices with limited resources when the edge workload is high. Hence, we design a collaborative local cache to balance this trade-off by caching results mainly from nearby users that are more likely to be reused, which effectively limits the size of the local cache and improves recognition accuracy.

Another key challenge is to establish a common coordinate system among users for utilizing the shared AR results and supporting the interactions. Otherwise, it is difficult to guarantee the consistency of the position and orientation of the annotation for the same augmented object perceived by different users. SEAR leverages a unique feature of collaborative AR that co-located users may be interested in some common object or scene for augmentation, which serves as a natural reference to synchronize the coordinate systems of different users. SEAR facilitates users to share their AR results and employs the local cache of shared results on mobile devices that contains these common objects/scenes for enabling fast coordinate synchronization. By doing this, SEAR significantly decreases end-to-end AR latency when the edge server is heavily loaded.

To incentivize users for sharing the results and thus improve the scalability of edge-assisted AR, SEAR empowers instantaneous interactions between neighboring users for *sharing AR experiences*. For users with overlapped viewports and perceiving the same annotation content, annotation manipulations from one user will be reflected on the screen of others in real time. For example, with SEAR users can play chess by freely moving the pieces on a virtual board [57, 69]. To smoothly synchronize these modifications, we design a bandwidth-efficient protocol that explores a key concept in the rendering pipeline for updating the position and orientation of annotations.

There are several advantages of SEAR's collaborative cache. First, it can alleviate the scalability issue of edge offloading by reusing compute resources on mobile devices. Second, it can lower the offloading cost by reducing traffic volume to the edge. Third, it offers a flexible and intelligent framework for developing interactive multi-user applications

to support the sharing of AR experience for users in the vicinity. Our experimental results show that SEAR reduces the AR latency to be less than 100 ms (on average) when serving hundreds of concurrent users, 130× faster than the state-of-the-art adaptive edge offloading solution [38], and brings down the uplink traffic by up to 33%.

We make the following contributions in this paper.

- We investigate the scalability issue of edge-assisted AR when serving multiple users, and measure the latency of experience-sharing features in a commercial AR SDK, which motivates the design of SEAR (§3) and draws the attention from developers of edge-native AR application to this critical issue.
- We design a collaborative caching mechanism and an efficient yet lightweight solution for reusing the shared results in the local cache of SEAR to balance the trade-off between latency and accuracy. SEAR not only scales but also shares AR experiences for hundreds of simultaneous users (§4). It lays the foundation to further optimize the scalability of edge-assisted mobile AR.
- We implement an interactive multi-user AR application atop a prototype for SEAR (§5) and conduct extensive evaluations using both real-world experiments and trace-driven simulations to demonstrate its efficacy and scalability (§6). The source code of SEAR is available at <https://github.com/xioage/sear>. The video of a SEAR demo is available at <https://youtu.be/TCdb6jYCyGU>.

2 BACKGROUND AND RELATED WORK

Background on Image Retrieval. AR systems can recognize objects inside personal displays (*e.g.*, the camera view of smartphones) using various image retrieval algorithms [49] and augment them with annotations [27]. A typical pipeline of image retrieval based mobile AR starts with *Object Detection* to identify the Regions of Interest (ROIs) in a camera view. After that, *Feature Extraction* is the second step that extracts the feature points [9, 41, 61] for each ROI. *Nearest Neighbor Search* finds the matching image in a reference database for the target ROI using its feature points. *Template Matching* validates the object-recognition result (*i.e.*, the nearest neighbor) and calculates the pose of the recognized object. *Object Tracking* is triggered to avoid object recognition for each of the consecutive frames. Finally, *Annotation Rendering* augments the recognized object by rendering the virtual annotation content associated with it. In this paper, we use image/scene retrieval and object recognition interchangeably.

Multi-user Collaborative AR has been investigated in the AR community for around two decades [13, 24, 32, 46, 56, 64]. More recent work proposed to share or cache previous AR results for improving socialization, recognition accuracy, user experience, and initialization delay [39, 54, 58, 71]. Among them, CARS [71] is our initial proposal to promote collaboration among nearby mobile users for improving their AR experiences. Edge AR X5 [58] utilized Web AR to solve the cross-platform issue for multi-user interaction synchronization and benefited from collaboration among users to reduce the initialization delay. Although existing work demonstrated the importance of collaboration in multi-user AR, there is limited effort on understanding and addressing the resulting scalability issue, especially when offloading AR tasks to edge servers. To the best of our knowledge, SEAR is the *first practical scheme that scales mobile AR experiences* for a large number of users.

Edge-assisted Mobile AR has drawn significant attention from the mobile computing community [16, 38, 62, 68, 70]. In our previous work Jaguar [70], we took advantage of GPU-accelerated edge servers to achieve low-latency and accurate object recognition for mobile AR. Liu *et al.* [38] enhanced the accuracy and reduced the latency of object detection in mobile AR by separating the rendering pipeline from edge offloading. Sel *et al.* [62] proposed analytic models to jointly optimize the cache size and energy consumption for edge-assisted mobile AR. Compared to the above work that optimized AR for *individual users* through computation offloading, SEAR addresses the critical scalability issue of edge-assisted mobile AR systems.

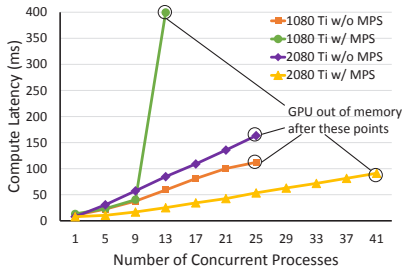


Fig. 2: Scalability of edge offloading for AR.

3 MOTIVATION

In this section, we motivate the design of SEAR by exploring the scalability of edge-assisted AR and measuring the performance of a commercial AR SDK.

3.1 Scalability of Edge-assisted AR

To understand the scalability of edge-assisted AR systems, we measure the server-side computation latency for different numbers of concurrent processes. For stress tests, each process handles *continuous* object-recognition tasks in AR using the image retrieval pipeline described above. The pipeline is accelerated by Nvidia GPUs with the CUDA parallel computing platform [45], as described in §5. We measure the performance of two GPU models, GeForce GTX 1080 Ti and RTX 2080 Ti (more powerful than 1080 Ti) that both have 11 GB memory.

We plot the computation latency per request for different configurations in Fig. 2. Every process handles 100 requests. Thus, the reported result for n concurrent processes is averaged over $100n$ experiments. The standard deviations are not shown in Fig. 2, as they are small for most data points. When running multiple applications on a single GPU, the CUDA kernels are executed sequentially in a round-robin fashion. CUDA offers the multi-process service (MPS) to better utilize compute resources and improve the scalability of GPU by reducing its context switching and on-GPU context storage. Hence, we compare the computation latency with and without MPS.

There are three key observations from Fig. 2. First, the maximum number of concurrent processes is 25 (13) for GTX 1080 Ti without (with) MPS, and 25 (41) for RTX 2080 Ti without (with) MPS, respectively. Both GPUs run out of memory after reaching these limits. Second, MPS indeed improves the scalability for RTX 2080 Ti. However, when using MPS for GTX 1080 Ti the computation latency drastically increases for more than 9 concurrent processes (likely due to a bug in the MPS implementation). Third, when running more than 40 concurrent processes on RTX 2080 Ti, the compute latency increases to around 100 ms (even with MPS). By considering the pre-processing and post-processing overhead on mobile devices and network latency, the end-to-end latency of edge-assisted mobile AR could be higher than 100 ms. However, it is known in the literature most AR systems require a less than 100 ms latency for a smooth user experience [14].

This scalability issue of edge offloading for AR applications motivates the design of SEAR, which benefits from a collaborative cache on mobile devices to reduce the workload on edge servers whenever it is feasible. We emphasize that some of the existing solutions include an adaptive offloading approach to reduce the number of AR tasks that will be processed on edge servers [14, 15, 38]. For example, Liu *et al.* [38] proposed to offload a frame to the edge only when it significantly differs from the most recently offloaded frame, which is determined by either high motion in the frame or a large number of changed pixels. Here, we use a simplified example with continuous offloading to exemplify the scalability issue and motivate our design. As we will show in §6, adaptive offloading can only *alleviate the scalability issue to some extent*, but may not be able to completely solve it, especially when the edge needs to serve hundreds of concurrent users. On the other hand, the larger the number of concurrent users is, the more opportunities SEAR may identify to utilize the collaborative cache on mobile devices for scaling AR experiences.

SEAR is orthogonal and complementary to solutions that optimize GPU acceleration on the edge. The above MPS from Nvidia is de-

signed to accelerate GPU, and the server-side image-retrieval pipeline of SEAR executes on top of MPS (§5). Moreover, we envision the edge will not be equipped with tens of GPUs for addressing the scalability issue due to economic reasons, given that the footprint of edge services should be much larger than the centralized cloud infrastructure.

3.2 Experience Sharing in Commercial SDKs

Both Apple ARKit and Google ARCore recently added the experience-sharing feature for supporting multi-user interactions [8, 20], which is related to what we plan to achieve in SEAR. Hence, we measure the end-to-end latency, energy consumption, and mobile data usage of ARCore's example Cloud Anchors application¹. It utilizes Google servers for hosting and resolving an anchor (*i.e.*, a fixed position and orientation in the real world). When hosting an anchor, it scans the environment of a mobile device and sends the visual mapping information to Google Cloud, which is processed into a sparse point cloud. When resolving an anchor, the mobile device sends the visual feature descriptors (*i.e.*, feature points) of its current camera view to Google Cloud, which matches these features to the generated sparse point cloud for the hosted anchor (*i.e.*, performing relocalization). Finally, Google Cloud sends back the detailed information of the anchor from the relocalization, which contains the orientation and the position.

We execute the application 40 times on a fully charged Samsung Galaxy S8 phone, 20 times for hosting anchors and another 20 times for resolving anchors. The network latency to Google Cloud is ~ 20 ms. Hosting an anchor takes considerably longer time and consumes more data than resolving it. For example, a user has to wait for ~ 10 (~ 0.44) seconds, since generating and processing 3D point cloud data is computation intensive, and uploads ~ 1.4 MB (29 KB) data when hosting (resolving) the anchor. The reason for the high data usage is that when hosting an anchor, the application may send multiple camera views, along with the data from motion sensors. Moreover, it also takes time for ARCore to initialize. The time taken from launching the application to finding a plane for augmentation is 3.508 ± 3.36 seconds. As all Cloud Anchors operations are allowed only after at least one plane is detected, the first hosting procedure actually finishes at least ~ 13.5 seconds after a user launches the application. Even the resolving of an anchor takes ~ 440 ms, much longer than the latency requirement (< 100 ms) for a smooth user experience [14].

We use the official Android profiling tool BatteryStats to measure the energy consumption of this application. The 40 operations (20 times each for hosting and resolving anchors) finish in about 17 minutes. After that, the phone's battery level drops from 100% to 91% (*i.e.*, a combination of hosting and resolving consumes ~ 13.5 mAh energy). Among the consumed energy, the application uses 74%, and the rest 26% is mainly used by the screen. The high latency and energy consumption of ARCore for supporting experience sharing in multi-user scenarios motivated us to design a lightweight scheme for sharing AR results and scaling AR experiences in SEAR².

4 SEAR SYSTEM DESIGN

In this section, we present an overview of SEAR and describe its key building blocks.

4.1 SEAR Overview

We show the client-side system architecture of SEAR in Fig. 3, which is decoupled into the App layer and the Base layer. The App layer receives camera views for object recognition and augmentation and tracks recognized objects. The Base layer handles object recognition and annotation synchronization for the App layer. This decoupled

¹The experience-sharing feature in ARKit leads to a higher latency due to its peer-to-peer communication nature and the compute-heavy data processing on mobile devices [53].

²Point cloud relocalization in a 3D space should be more generic than the method we use in SEAR, as it can provide more information to applications (*e.g.*, the depth of an object). However, we believe that it may be an overkill for the scenarios that we focus on in this paper, which have limited benefit from the depth information. We are optimizing point cloud relocalization in another ongoing project.

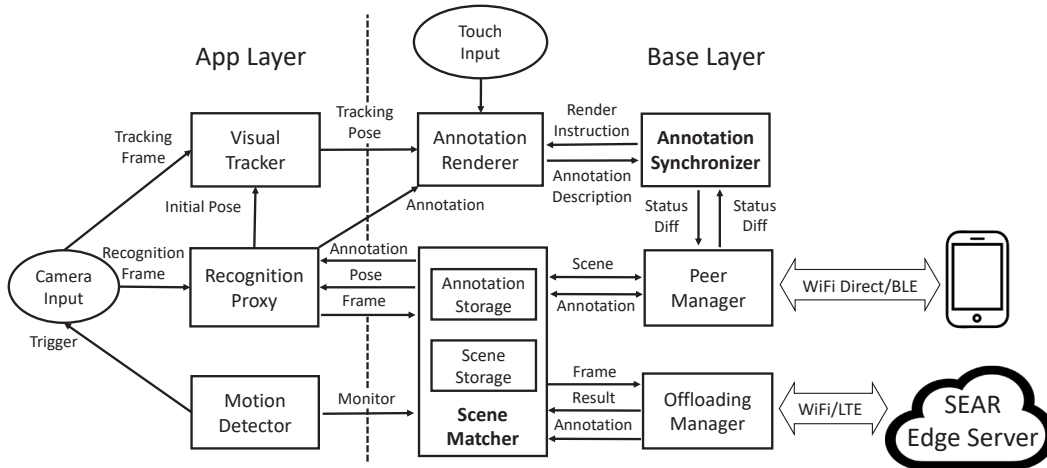


Fig. 3: System architecture of the SEAR framework (client side).

framework enables the App and Base layers to evolve independently of each other, making the development of AR systems easy. We design the SEAR server by following the pipeline described in §2.

The App layer has three components: *Recognition Proxy*, *Visual Tracker*, and *Motion Detector*. *Recognition Proxy* bridges the App and Base layers for object recognition. Its input is a camera view and the output contains the pose of the recognized object in the view. *Visual Tracker* tracks the pose of the recognized object when the camera view changes. It is initialized by *Recognition Proxy* and takes continuous camera views as input. We utilize feature points to calculate object pose in a frame and optical flow tracking [25] to track feature points in a frame sequence, avoiding performing object recognition for each frame. Features may be points or edges in the image, and may also be the result of a feature extraction algorithm such as SIFT (Scale-Invariant Feature Transform) [41] and SURF (Speeded Up Robust Features) [9] applied to the image. *Motion Detector* provides a seamless user experience for triggering object recognition without specific user commands such as screen touch or voice input. To ensure object-recognition accuracy, it continues monitoring the user's hand movement and sends this information to *Scene Matcher* of the Base layer. Object recognition will be performed when a mobile device is in a relatively steady status, which filters out blurry images.

The Base layer has five components: *Annotation Renderer*, *Scene Matcher*, *Annotation Synchronizer*, *Peer Manager*, and *Offloading Manager*. *Annotation Renderer* augments the recognized object using an annotation content and calculates its pose for rendering. It reacts to users' input and instructions from *Annotation Synchronizer* for modifications from others and renders annotations accordingly. *Peer Manager* utilizes WiFi Direct or Bluetooth Low Energy for peer discovery and the sharing of AR results. *Offloading Manager* communicates with the edge server for submitting recognition requests and receiving the result and annotation.

4.2 Offloading Decision

The key challenge of making offloading decisions is to determine *where an object-recognition request should be processed*. With the goal of optimizing end-to-end latency, SEAR dynamically makes the decision based on the estimated edge workload, and thus tackles the scalability issue when serving recognition requests from multiple concurrent users. When the edge workload is low, offloading compute-intensive tasks may reduce the overall latency. However, if the edge workload is high, blindly offloading these tasks may lead to a higher latency than executing them on mobile devices by utilizing the local cache.

Intuitively, SEAR should make offloading decisions on the edge server, because the decision is mainly determined by its workload. Upon receiving an object-recognition request, the edge either pushes it into its task queue and returns the result once it is available, or immediately responds to the mobile device with a rejection message if its estimated workload is high. However, this method is preferable only

when the latency between the mobile device and the edge is extremely low (e.g., < 1 ms). Otherwise, SEAR needs to bear with extra network latency for sending the request and receiving the response, which may not be neglectable compared to the computation latency reduced by offloading. SEAR can also search local cache first before making the offloading decision. However, our experimental evaluation indicates that local search may take ~ 100 ms (§6.2). Thus, we should resort to the local cache only when the estimated edge workload is high.

Given that latency should be the first-class citizen for mobile AR, SEAR makes offloading decisions on mobile devices and takes advantage of another option that periodically sends the edge workload to AR applications. It offloads AR tasks to the edge *only if the estimated offloading latency is lower than the latency of executing these tasks by using the cached results* (suppose a cache hit will happen). When a cache miss occurs due to the limited cache size, SEAR has to offload the recognition request to the edge with a high priority for reducing the queuing delay. With this approach, SEAR can also reduce the uplink traffic by avoiding the transmission of requests that may be rejected by the edge. We can optimize the overhead of delivering edge-workload messages through application-layer broadcast where a device forwards the received messages to others. SEAR can send the workload information along with application-specific data to further reduce its overhead.

The edge server maintains two queues, one for normal-priority tasks and another for high-priority tasks that are caused by cache misses on mobile devices, and creates a process pool to handle all the requests. For each process, when the current recognition task is finished, it will fetch a task from the high-priority queue if this queue is not empty. Otherwise, it handles the tasks in the normal-priority queue. There are several methods for estimating the edge workload. For example, we can use the `nvidia-smi` management interface to check the utilization of Nvidia GPUs and their associated memory. Due to the importance of latency for AR applications, SEAR determines the edge workload based on the *computation latency*, which includes both the processing time of the scene retrieval pipeline and the queuing delay on the edge. We can use the most recent measurement result or the running average over the previous N executions for estimating the queuing delay. We can also build a model to calculate the queuing delay, by considering both the high-priority and normal-priority queues. We will evaluate the performance of these methods in §6.3.

4.3 Scene Matcher

We now describe the workflow of *Scene Matcher*. Upon receiving a recognition request and if the edge workload is high, it tries to find a match in the local cache. It first computes the feature descriptors and their compact representation for the current camera frame. It then calculates the distance between this compact feature representation and those in *Scene Storage* (i.e., the local cache) to see whether it can find a match. *Scene Matcher* determines the homography between

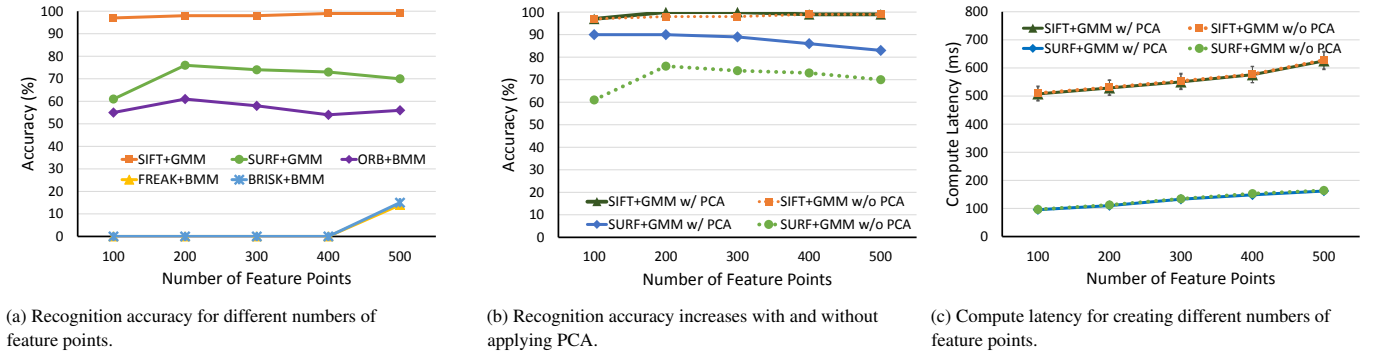


Fig. 4: Performance comparison of various feature extraction algorithms on Samsung Galaxy S9.

the features of the camera frame and those of the local match to get their geometric transformation. Finally, it adjusts the corresponding recognition result (e.g., the bounding box of recognized objects) from the local match accordingly for rendering.

An important design decision for Scene Matcher is that we utilize the combination of a *kernel function* and a *probabilistic model* to encode the feature descriptors, which accelerates object recognition on mobile devices and reduces the memory consumption of cached results. Object recognition is slow when comparing raw feature descriptors directly, because their numbers may vary for different images. Specifically, we use Fisher Vector (FV) [49] with Bernoulli Mixture Model (BMM) [30] for binary features and with Gaussian Mixture Model [59] for non-binary features. FV is a compact feature representation for encoding image features such as SIFT [41], SURF [9], and ORB [61]. It is a special and improved case of the general Fisher kernel [50]. A mixture model can learn the subpopulations within an overall population automatically without knowing which specific subpopulation a data point belongs to. For example, GMM is a probabilistic model for representing subpopulations that are normally distributed. For BMM, those subpopulations should follow a Bernoulli distribution.

The Fisher Vector encodes feature points and their mean and covariance deviation vectors from the probabilistic model using a compact representation. After the encoding, each image is represented as a *{frame image, recognition result, feature points, compact feature representation}* tuple for accelerating the local search. We select up to k nearest neighbors from the local cache for the final verification (i.e., template matching in the pipeline in §2). When a cache miss happens, the recognition request will be forwarded to the edge server with a high priority, for reducing the queuing delay on the edge and optimizing the end-to-end latency. The cache size may affect the end-to-end latency. Suppose the latency for local execution by searching the cache is T_l and the latency for remote execution by offloading to the edge is T_r . The expected end-to-end latency will be $T_l \times (1 - \alpha) + T_r \times \alpha$, where α is the cache miss ratio. On the positive side, a large cache size may reduce the miss ratio α and thus the edge workload. On the negative side, a large cache will increase the search time of the local cache T_l .

4.4 Feature Extraction in SEAR

A key design challenge of SEAR is the selection of a *lightweight yet efficient and suitable* feature extraction algorithm for balancing the trade-off between accuracy and latency. SIFT [41] and SURF [9] were used before in mobile AR systems (e.g., SURF in OverLay [27] and SIFT in VisualPrint [28]). SIFT is a widely applied algorithm to describe local features in images. SURF is faster than SIFT and is robust against different image transformations. Instead of applying them directly for SEAR, we evaluate the performance of five well-known feature extraction algorithms, SIFT, SURF, ORB [61], FAST+FREAK [5], and BRISK [36] (the last three generate binary features). We evaluate their performance with two key metrics, computation latency of the entire image retrieval pipeline and object-recognition accuracy, which is defined as the ratio between the number of successful recognition requests (on a reference-image dataset) and the total number of requests.

We plot the experimental results for the public Paintings dataset [3] in Fig. 4. We randomly select 1,000 images from this dataset for

our experiments³. The results are averaged over 100 testing images captured from the original for emulating typical AR use cases (i.e., they are used as camera views). We use FV+GMM to create the compact representation for SIFT and SURF because they generate non-binary feature points, and FV+BMM for others as they produce binary features. The results in Fig. 4a show that SIFT achieves the highest accuracy, followed by SURF. The accuracy of the other three algorithms is low. Even for ORB that performs the best among the three, its accuracy is lower than 70%. Hence, we exclude these three algorithms from the candidate list.

We next apply principal component analysis (PCA), a method that reduces the dimension of feature points, to SIFT and SURF, which is known for improving the performance of feature extraction algorithms [31]. The results in Fig. 4b show that PCA can significantly boost the accuracy of SURF, especially when using 100 feature points (61% vs. 90%). The improvement on SIFT is limited as its accuracy is already high (close to 100%). The accuracy of feature extraction algorithms also depends on other factors such as the size of the dataset. When using a smaller dataset with 100 images, the accuracy of SURF with 100 feature points could be as high as 98%. One interesting observation from Fig. 4b is that the accuracy of SURF decreases when more feature points are generated regardless of whether we apply PCA or not. This is counter-intuitive. In general, the accuracy should be higher when more feature points are extracted [17]. One possible reason is that our testing images are captured from the original ones to emulate typical AR use cases. The extracted feature points from a captured image cover also content in the camera view that is outside of the original image. The more feature points we extract, the more these noisy points would be, decreasing recognition accuracy. We verify that the accuracy will not drop when directly using the original images as testing images. To address this problem, we can first perform object detection and then extract feature points for only the detected object(s).

We finally measure the computation latency of SIFT and SURF with different numbers of feature points on Samsung Galaxy S9. We plot the average and standard deviation of the results for 100 experiments (one for each testing image) in Fig. 4c. The main observation from this figure is that in general creating more feature points increases the computation latency. SIFT leads to a latency that is higher than 500 ms even with only 100 feature points. The computation latency of SURF ranges from ~80 ms to 160 ms, depending on the number of feature points. Fig. 4c also shows that applying PCA to SIFT and SURF has almost no impact on the latency. The results in Fig. 4 are for experiments using only the nearest neighbor for the local search. Considering more nearest neighbors may slightly improve the accuracy, but will increase computation latency because we need to execute template matching multiple times (one for each neighbor). We validate the above results with another public dataset, the CMU-Oxford Sculpture [1], using the same setup as the Paintings dataset. We observe similar trends as those shown in Fig. 4. The accuracy is close to 100% for SIFT with 100 feature points, regardless of whether PCA is applied or not. It is 93%

³A dataset with 1,000 images is reasonably large. In the literature, OverLay [27] evaluates the performance of feature extraction algorithms with 100 images and VisualPrint [28] uses 500 images.

(59%) for SURF with 100 feature points and when PCA is (not) applied.

To summarize, given the results in Fig. 4, we use SURF with 100 feature points and the nearest neighbor search for SEAR, which achieves the best trade-off between compute latency and recognition accuracy. We will show in §6.2 (Fig. 5) that when working on a collaborative local cache in SEAR with only 100 items (instead of the above dataset with 1,000 images) SURF achieves close to 100% recognition accuracy.

4.5 Annotation Synchronizer

Annotation Synchronizer enables instantaneous interaction for collaborative users who are in proximity of each other. It synchronizes the annotation status so that one can see the modifications made by others. The key challenge here is the *synchronization of changes in real time*. A naive solution is to share users' touch input directly. This is not feasible since users may view the scene from different angles, and the same touch input from different viewing angles could lead to different transformations in 3D space. We explore an important concept in the rendering pipeline, *Model Matrix* (i.e., object-to-world matrix), to design a lightweight scheme. In a rendering system, this matrix transforms the position of an object in its own model to the world space. Therefore, rendering is affected by a combination of the transformations of the annotation that is being drawn. Manipulations from a user change only the Model Matrix of annotations within the user's local world. Recognition and tracking of common real-world objects guarantee that the coordinate systems (i.e., world space) of the overlaid annotations for different users are *synchronized* through the coordinate system of the physical world (as the reference here). As a result, we can exchange this Model Matrix between users for synchronizing annotation modifications. The persistence of annotation modification is enabled by the edge server. When a user finishes modifying the annotation, SEAR uploads the final annotation status to the edge. Later on, when the corresponding scene is requested by the same user or others, the edge can send the stored annotation status to them.

At the beginning of annotation synchronization, Peer Manager communicates with others to check if they are looking at the same objects by exchanging the object IDs included in recognition results. After that, it shares the description of annotations (e.g., a 3D object's position, scaling, etc.) generated by Annotation Renderer. When a user's touch input leads to any status change (e.g., translation or scaling) on any object in the renderer, SEAR serializes the changes and sends them in a packet to the peers, which includes the annotation object ID and the object's updated Model Matrix. This synchronization is continuous so that the peers could see fluent modifications instead of just final results. It is worth mentioning that the Model Matrix for each object is as simple as 16 float numbers, making it lightweight for real-time peer-to-peer transmission. SEAR currently supports moving and scaling operations from users, and our design is capable of extending to more complex operations. SEAR utilizes a simple mechanism to resolve conflicts of operations to the same annotation content. When an annotation is being modified by a peer and Peer Manager continuously receives the updated Model Matrix of the annotation, Annotation Synchronizer stops receiving the touch input for that specific object. This prevents conflicts from happening. However, simultaneous manipulations on different annotations from different users are still feasible, since these operations would not lead to any status conflict. We can extend this design for more complicated scenarios by using existing strategies (e.g., the one proposed by Gautier *et al.* [19]).

5 IMPLEMENTATION

We build a prototype of SEAR, using several open-source software packages, and implement an interactive multi-user AR application on top of it. The SEAR client is an Android application and uses *VLFeat* [65] and *OpenCV4Android* [48]. Based on the results in §4.4, we select the SURF [9] feature extraction algorithm, Fisher Vector [49], and PCA to construct the GMM [59] model offline. The size of the model is only 15 KB for our setup and does not depend on the number of images in the dataset. We use the gyroscope and accelerometer on mobile devices to analyze users' hand motion for Motion Detector. We empirically set the local cache to have 100 entries, which achieves

Video	BarScene	Boxing	Crosswalk	Driving
Frequency (s)	6.8	2.8	1.9	2.6
Video	FoodMarket	Narrator	RollerCoaster	Square
Frequency (s)	1.9	3.4	1.6	2.8

Table 1: Frequency of sending object-recognition requests for the adaptive edge offloading scheme proposed by Liu *et al.* [38].

a good balance between the computation latency and the recognition accuracy. Each cached item takes about 40 KB of memory space, which leads to an overall memory consumption of 4 MB. To further optimize the computation latency of scene retrieval on mobile devices, we limit the number of recognized objects to be one. Thus, SEAR supports the augmentation of one object in the camera view at a time.

The SEAR server implements the image/scene retrieval pipeline using *OpenCV* [47], *VLFeat* [65], and *CUDA* [45]. We use the SIFT feature extraction algorithm due to its higher recognition accuracy than SURF. We generate SIFT feature points and apply PCA to them for dimension reduction. When extracting SIFT feature points on the server, the computation latency is not a major concern (only 2.89 ms). We first utilize the GPU implementation of SIFT feature extraction by Björkman *et al.* [11] to generate feature points. We then employ the *VLFeat* library to train offline the GMM model that represents a statistical distribution of feature points. Finally, we create the Fisher Vector from the feature points of each image using the GPU implementation by Ma *et al.* [42].

6 EVALUATION

In this section, we evaluate the performance of SEAR through both real-world experiments and trace-driven simulations (as it is difficult to conduct experiments with hundreds of users).

6.1 Experimental Setup

Testbed. Our testbed consists of several clients and an edge server. The clients are Samsung Galaxy S9 (SGS9) smartphones (Snapdragon 845 processor, octa-core 4×2.8 GHz & 4×1.7 GHz). The edge server is equipped with Intel I7-5820K CPU, 32GB memory, and Nvidia GTX 1080 Ti GPU. We create a WiFi hotspot on the edge server and connect mobile devices to the hotspot over 802.11g at 2.4 GHz. With the WiFi interface connected to both an access point in the Station mode for Internet access and a peer device in the WiFi Direct mode, the bandwidth is ~32 Mbps between the phones (using WiFi Direct). When we connect the phone to the Internet via cellular networks, the WiFi Direct bandwidth increases to ~60 Mbps, as it can use all the airtime for WiFi Direct (no sharing with the Station mode anymore). The RTT between the phones when using WiFi Direct is only ~6 ms. Note that we use this testbed with a single edge server and our prototype implementation to evaluate the recognition accuracy and break down the end-to-end latency for SEAR.

Trace-Driven Simulator. In order to evaluate the effectiveness of SEAR on scaling AR experiences, we build a trace-driven simulator that uses the latency results measured on our testbed for its various components. One challenge is to identify a suitable mobility trace with a large number of users. Ideally, we should use traces collected from social contacts detected by, for example, Bluetooth scanning. However, the size of most publicly available datasets in this category is small (with less than 100 users). We resort to the student contact patterns derived from campus schedules of the National University of Singapore (NUS) [63]. The original dataset has the class schedules of 22,341 students attending 4,486 courses during 76 time slots in one week. We generate a mobility trace from this dataset to simulate the scenario where 447 students visit an art museum of 1,000 paintings. We treat a course session as visiting a painting for a minute. Hence, the duration of our simulation is 76 minutes. Although this synthesized mobility trace may not be as realistic as those collected via Bluetooth scanning, it is the best we can do for this large scale. This synthesized mobility trace is publicly available at <https://github.com/xioage/sear/>

To determine the frequency of sending object-recognition requests to the edge, we implement the adaptive offloading scheme proposed by Liu *et al.* [38]. We show the results in Table 1 for the same dataset with eight videos. For videos with high motion, even the adaptive offloading

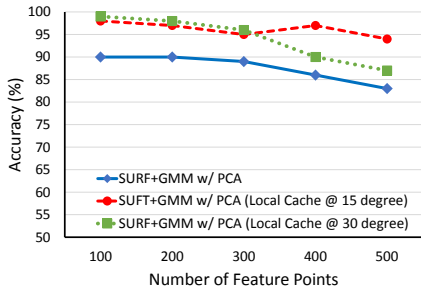


Fig. 5: Recognition accuracy for local caching on mobile devices.

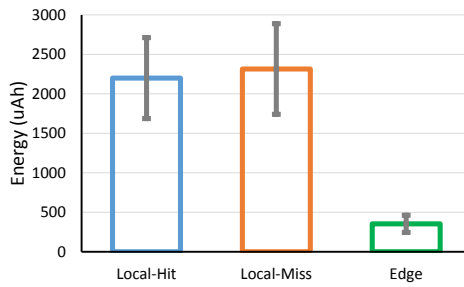


Fig. 6: Energy consumption for scene retrieval on mobile devices.

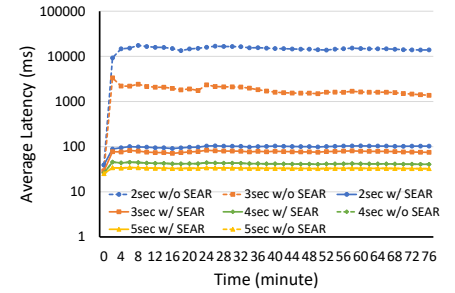


Fig. 7: Average latency w.r.t. different offloading frequencies.

frequency could be high. For example, we need to send an offloading request *every 1.6 seconds* on average for the RollerCoaster video. As we will show in §6.3, using the adaptive offloading alone, this high request frequency will lead to scalability issues when serving hundreds of concurrent users.

6.2 Results from Testbed Experiments

We evaluate the performance of SEAR on our testbed using the following metrics: recognition accuracy, scene retrieval latency, energy consumption on mobile devices, and synchronization latency of annotation content, and compare it for the first two metrics with CARS [71], our previous work that shares object-recognition results from the cloud to facilitate socialization among users for collaborative AR.

6.2.1 Recognition Accuracy

We measure the recognition accuracy of scene retrieval on both mobile devices and the edge server. Note that we measure this accuracy under cache hit scenarios for the local cache on mobile devices. We will evaluate the cache hit ratio in §6.3. When a cache miss happens, there is no way for SEAR to correctly recognize the target object using the cached results. It will have to forward the recognition request to the edge. Since the last step of the scene retrieval pipeline is feature matching, it effectively filters out the false-positive results. Therefore, the reported recognition results are all true positives.

We plot the recognition accuracy of applying the image retrieval pipeline to the local cache (which is used as the reference-image database) in Fig. 5 and compare it with the recognition accuracy reported in §4.4 (“SURF+GMM w/ PCA” in Fig. 4b). The key difference is that when evaluating the accuracy of various feature extraction algorithms in §4.4, we apply the image retrieval pipeline to the dataset with 1,000 original images. However, the images in the local cache are the camera views of the original images, potentially from different viewing directions. To evaluate the impact of viewing-angle differences on recognition accuracy, we capture 100 images (randomly selected from the dataset) with a viewing direction orthogonal to the image and use them as cached items. We then capture these 100 original images again from different viewing angles and feed the captured images to the image retrieval pipeline (over the local cache) for recognition. We plot the results in Fig. 5 for two viewing directions, 15° and 30° away from the one orthogonal to the image. The latter (30°) serves as the worst-case scenario. In practice, users tend to augment objects/scenes when facing them directly.

Fig. 5 shows that regardless of viewing-angle differences, SEAR achieves close to 100% recognition accuracy when extracting 100 SURF feature points, by *limiting the size of the local collaborative cache* to be 100. Recall that a smaller dataset results in higher recognition accuracy. As shown in §4.4, the recognition accuracy of SURF on a larger dataset with 1,000 images drops to ~90%. We repeat the above experiment using different groups of randomly-selected 100 images and observe qualitatively similar results. The recognition accuracy for image retrieval on the edge using SIFT features is 100%, which is reported in Fig. 4b. Hardware acceleration on the edge only reduces computation latency and has no impact on recognition accuracy.

Step	Latency (ms)
Pre-processing at Client	6.530 ± 1.19
Comm. between Client & Edge	12.47 ± 3.62
Image Retrieval at Edge	12.85 ± 1.53
Post-processing at Client	2.760 ± 0.58

Table 2: Breakdown of end-to-end scene retrieval latency for edge offloading (averaged over running experiments 20 times). The value after $\pm$ is the standard deviation.

Step	Latency (ms)
SURF Feature Extraction	83.15 ± 6.25
Fisher Encoding with GMM	1.350 ± 0.12
Nearest Neighbor Search	1.250 ± 0.15
SURF Feature Matching	1.420 ± 0.26
Pose Calculation	8.380 ± 1.29

Table 3: Breakdown of averaged latency (ms) for different AR tasks on mobile devices. The value after $\pm$ is the standard deviation (for running experiments 20 times).

6.2.2 Scene Retrieval Latency

We next focus on the scene retrieval latency of SEAR when there is only a single user. We will present the latency results for multiple concurrent users in §6.3. The latency is defined as the time from generating an object-recognition request to displaying the corresponding annotation on the mobile device. When offloading AR tasks to the edge, this end-to-end latency includes the processing time on both the mobile device and the edge and the round-trip network transfer time.

We start with measuring the overall latency for edge offloading. We break it down into several steps and list the latency for each step in Table 2, which reports the average results from running the experiments 20 times. The end-to-end latency is less than 35 ms for edge offloading (the best-case scenario with a single user). Among them, image retrieval on the edge takes only 12.85 ms, because it leverages GPUs to accelerate the pipeline. We also measure the latency when GPU acceleration is not available, which would increase by at least 160 ms. For other steps, pre-processing on the client shrinks the size of a frame, converts it to grayscale, and encodes it into a JPEG file. It takes less than 7 ms. Post-processing calculates the pose of the annotation for the recognized object in less than 3 ms.

We then measure the compute latency for the image retrieval pipeline that takes advantage of collaborative caching on mobile devices. It involves five steps, and we list their completion time in Table 3. As we can see from this table, most of the time is spent on extracting the SURF feature points, which consumes ~83 ms (86% of the overall latency). Recall from §4.4, SEAR can achieve high accuracy when matching with only the nearest neighbor (*i.e.*, executing feature matching only once). Hence, the image retrieval latency on the mobile device is ~96 ms. Note that this result is for the cache hit scenario. When the cache miss occurs, SEAR will have to offload the object-recognition request to the edge with a high priority. In this case, the end-to-end latency increases to ~130 ms, which includes both the local computation latency and the latency for handling the request offloaded to the edge. We will evaluate the cache hit ratio in §6.3.

To summarize, the latency that SEAR can achieve is usually under 100 ms, which dramatically outperforms CARS [71] whose best-case latency is ~ 160 ms. Benefiting from SEAR's collaborative local cache, its recognition accuracy is close to 100% when using SURF features, which is much higher than the 60% accuracy achieved by CARS [71]. Moreover, the end-to-end latency of SEAR is much lower than that for hosting and resolving an anchor in ARCore (>10 seconds for hosting and ~ 440 ms for resolving in §3). The key reason is that SEAR avoids point cloud based relocalization which is time consuming. Instead, it resorts to objects in the overlapped viewports of nearby users and takes them as the reference to synchronize the coordinate systems of these users. Another reason is that SEAR consumes less data than ARCore (~ 10 KB for recognition request and scene retrieval result in SEAR vs. more than 1.4 MB in ARCore).

6.2.3 Energy Consumption & Resource Utilization

We measure energy consumption on mobile devices for both cache hit and cache miss of the collaborative caching scheme and edge offloading scenario using the Monsoon Power Monitor (<http://monsoon.github.io/powermonitor/>) and Samsung Galaxy Note4, as the power monitor requires the device to have a removable battery that Samsung Galaxy S9 does not support. When offloading computation-heavy tasks to the edge, since the Note4 is the only user, its requests will be immediately processed by the edge without any queuing delay, which represents the best-case scenario.

We show the measurement results in Fig. 6. There are two key observations from this figure. First, local scene retrieval consumes more energy than the best-case scene retrieval at the edge, mainly because it conducts all compute-intensive tasks on the mobile device. Moreover, the local scene retrieval takes a longer time to finish than offloading to the edge, during which the screen is always on and consumes energy. However, when the workload on the edge is high, the end-to-end latency for offloading will increase, which leads to a longer waiting time on the client and thus higher energy consumption than what is shown in Fig. 6. Since the screen usually consumes less energy than AR tasks, we expect the energy consumption for the worst-case scene retrieval at the edge is still lower than that of executing all tasks locally on the mobile device. The second observation is that a cache miss leads to higher energy consumption than a cache hit. The reason is that the end-to-end latency will be higher when the cache miss happens, as it requires offloading the recognition task to the edge, during which the screen consumes energy. Overall, benefiting from the collaborative caching design, the energy consumption of SEAR (<2.5 mAh) is much lower than that of ARCore (~ 13.5 mAh in §3). We measure the computation resource utilization of SEAR on a Samsung Galaxy S9 smartphone. The CPU utilization of SEAR is around 80% during object recognition, which is acceptable for modern mobile devices with multiple CPU cores. The memory consumption of SEAR is only around 170MB.

6.2.4 Annotation Synchronization

The performance of annotation synchronization is important in such a multi-user AR system, as delayed or unsmooth annotation manipulations in a peer's camera view may significantly affect the user-perceived QoE. To understand the performance of our annotation synchronization scheme, we break down its end-to-end latency, which is defined as the time period from a user's touch input to the peer's generation of annotation modification instruction. We can divide it into four steps, as shown in Table 4. The latency of annotation synchronization between two users with WiFi Direct is merely 7.68 ms on average.

We also measure the rate of annotation manipulations, in terms of frames per second (FPS), which affects the smoothness of these modifications perceived by users. During our experiments, the annotation synchronization happens at ~ 59 FPS, which is high enough for smooth motion. It is worth mentioning that the bandwidth consumption of annotation synchronization is far below the limit of WiFi Direct. The transmission of the updated Model Matrix for each modified object requires only 4.69 KB/s of data usage.

Step	Latency (ms)
Touch to Instruction	1.09 ± 0.31
Instruction to Transmission	1.98 ± 0.72
Transmission	3.24 ± 1.56
Reception to Instruction	1.37 ± 0.44

Table 4: Breakdown of averaged latency for different annotation synchronization tasks. The value after $\pm$ is the standard deviation (for 20 experimental runs).

6.3 Results from Trace-Driven Simulations

Using the trace-driven simulator described in §6.1, we measure the performance of SEAR in terms of scalability, uplink traffic, the effectiveness of different edge-workload estimation methods, and cache hit ratio, and compare the scalability of SEAR with the adaptive edge offloading scheme [38].

6.3.1 Scalability

We evaluate the scalability of SEAR for four different offloading frequencies, that is, sending an object-recognition request every 2, 3, 4, and 5 seconds. We select these values based on the analysis of the adaptive offloading scheme proposed by Liu *et al.* [38] (Table 1 in §6.1). We show the end-to-end latency in logarithmic scale with and without SEAR in Fig. 7. The latency without SEAR is essentially the scalability performance of Liu *et al.* [38] in the multi-user scenario.

There are three main observations from Fig. 7. First, when using the state-of-the-art adaptive edge offloading approach [38] for multiple concurrent users, the end-to-end latency will substantially increase if the required offloading frequency is high. When mobile devices send a request every 2 seconds on average (*e.g.*, required by the Crosswalk and FoodMarket videos in Table 1), the end-to-end latency will be higher than 10 seconds, which may significantly affect the user-perceived QoE. Second, SEAR can effectively address the scalability issue when serving more than 400 users simultaneously by using the local compute resources on mobile devices whenever it is feasible. Even for the two-second offloading frequency, SEAR can maintain an average end-to-end latency of ~ 100 ms, $130\times$ faster than using alone the adaptive edge offloading solution [38]. Given this huge gap, simply increasing the computing capability of the edge will not be an efficient solution to address the scalability issue. Third, when the offloading frequency is low, for example, offloading a request every 4 seconds on average or longer, there is almost no difference in the end-to-end latency between SEAR and the adaptive edge offloading. The reason is that for both schemes almost all object-recognition requests will be handled by the edge server that is not overloaded (given the low offloading frequency).

6.3.2 Uplink Traffic

We next compare the amount of uplink traffic consumed by SEAR and the adaptive edge offloading solution [38]. SEAR can potentially reduce uplink traffic by executing the scene retrieval pipeline locally on mobile devices. We show the amount of accumulated uplink data usage (in GB) in Fig. 8 for different offloading frequencies. As expected, the amount of uploaded data will increase for a higher offloading frequency. When sending the requests every 2 seconds on average, SEAR can reduce the uploaded data by around 33% (8 GB vs. 12 GB), which lowers the cost of offering mobile AR services. There is almost no traffic reduction for low offloading frequencies (*e.g.*, sending a request every 4 or 5 seconds on average). In this case, even if there is a cached item that matches the recognition request, SEAR prefers to offload it to the edge, which leads to a lower latency due to the low offloading frequency and thus the low edge workload.

6.3.3 Estimation of Edge Workload

As mentioned in §4.2, SEAR can estimate the edge workload using various schemes. We show their impact on latency in Fig. 9 for two offloading frequencies (*i.e.*, sending an object-recognition request every 2 or 3 seconds). We have two observations from this figure. First, the performance of using the latest measurement result is close to that of using the running average of the previous 20 executions. Second, the above two schemes perform slightly better than the approach that

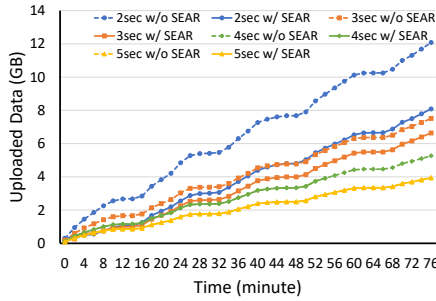


Fig. 8: Total uploaded data w.r.t. different recognition frequencies.

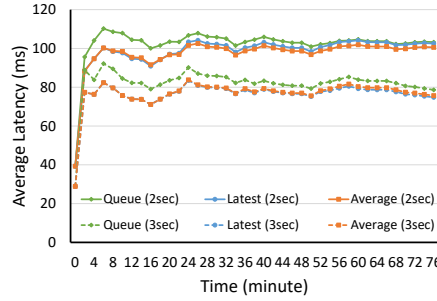


Fig. 9: Average latency for different estimations of server workload.

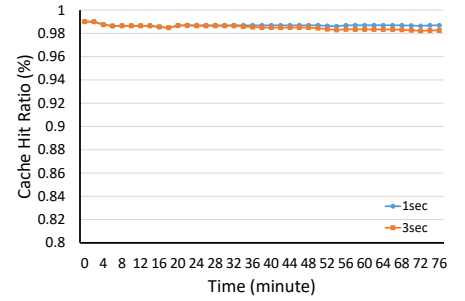


Fig. 10: Cache hit ratio for local caching on mobile devices.

models the queuing delay, because our model may underestimate the waiting time of concurrent requests in the task queue.

6.3.4 Cache Hit Ratio

We finally measure the cache hit ratio of SEAR for two sharing frequencies, exchanging cached items with peers every second and every 3 seconds. We show the experimental results in Fig. 10. As expected, more frequent exchanges of cached items will (slightly) increase the cache hit ratio. As we mentioned in §6.2, when a cache miss happens, the end-to-end latency increases to ~ 130 ms. Fig. 10 demonstrates that the cache hit ratio of SEAR is higher than 98% (thanks to the sharing opportunities offered by hundreds of concurrent users), which means that users will only occasionally experience this high latency. Note that the cache hit ratio may be affected by factors such as the number of users. Initially, with a large number of users, the cache can be quickly filled, improving the hit ratio and reducing end-to-end latency. When the cache is full, the impact of the number of users on the cache hit ratio depends on the replacement policy. We implement a policy based on the popularity of the images whose distribution follows Zipf's law [12]. A user fetches the most popular but missing item from the peers for replacing the most unpopular item in the local cache.

7 DISCUSSION AND FUTURE WORK

In this section, we discuss several open issues and research opportunities for further exploration.

Deep-learning Based AR. In this paper, we focus on AR systems that take advantage of image retrieval for object recognition [27, 28, 70], instead of *object classification* based solutions [7, 38] that leverage deep neural networks such as ResNet [23], AlexNet [33], and YOLO [55] to learn features. Compared to them, image retrieval based object recognition can offer more contextual information to AR. For example, during sightseeing, object classification based AR will simply label a landmark (e.g., the Empire State Building) as a skyscraper, but image retrieval based AR [27, 28, 70] can offer visitors a detailed description of the landmark after recognizing it. Given that object classification aims to classify images into different categories, (e.g., 1,000 classes for the ImageNet [2] database), it is difficult to perform an apple-to-apple comparison of these two types of systems (*i.e.*, they tackle different problems and have different use cases). However, it is feasible to extend the high-level idea of SEAR to deep-learning based AR systems, which is our ongoing work.

Location-facilitated AR. SEAR is not limited to indoor environments and *does not require localization*. It is also helpful for outdoor scenarios (e.g., city sightseeing). Localization alone is not sufficient for AR. For example, even if we know which painting a user is visiting in a museum via localization, we cannot tell the position and orientation of that painting within a camera view for augmentation. Another possible solution for reducing the edge workload is to preload all image features on mobile devices and store them with room-level partitioning to avoid the impact on latency and recognition accuracy. However, room-level localization [10] will increase computation overhead and energy consumption on mobile devices.

Sharing Policy. One practical issue of SEAR is to determine with whom to share the recognition results and annotations that are stored

in the local cache. This problem has been well studied in the literature of mobile social networks [26, 37]. For example, SEAR can optimize the sharing policy via preference-aware schemes such as PrefCast [37], which reduces the chance of exchanging objects that other users may not be curious about. We leave the integration of these existing approaches into SEAR as our future work.

Security & Privacy have always been key issues for mobile applications and AR makes no exception [29, 34, 35, 60]. For example, Recognizer [29] is an OS abstraction that offers fine-grained access control to AR applications (e.g., exposing a skeleton, instead of raw sensor data). Lebeck *et al.* [34, 35] study the output management of AR systems, focusing on safety-related issues (e.g., augmenting on windshield without obscuring oncoming vehicles). We argue that privacy issues may not exist for classroom education [51], military training [40], and most enterprise use cases such as industrial field services, collaborative design and modeling, *etc.* These use cases can all benefit from SEAR. We plan to address the security and privacy concerns for other use cases by taking advantage of the plethora of existing work in the literature.

Other Open Issues. We use a single metric for estimating edge workload in SEAR. We should also consider other factors such as network conditions and energy consumption on mobile devices, and make the offloading decision via an optimization framework [52]. The scalability of SEAR is demonstrated through mainly trace-driven simulation. To better understand the performance of SEAR in the wild, we plan to conduct large-scale experiments by deploying SEAR in places such as museums and galleries after addressing the above issues.

8 CONCLUSION

In this paper, we propose SEAR, a first-of-its-kind collaborative framework for scaling multi-user AR experiences when offloading computation-intensive tasks to the edge. SEAR leverages the unique feature of AR that nearby users may be interested in common objects for augmentation to synchronize their coordinate systems and share the AR results among them, which helps address the scalability issue of edge-assisted mobile AR systems. We make SEAR feasible through the utilization of a collaborative cache on mobile devices for balancing the trade-off between latency and accuracy. We build a proof-of-concept for SEAR and demonstrate it can achieve not only high object-recognition accuracy but also a significant reduction of AR latency when the edge server is overloaded. Given the wide range of use cases for mobile edge computing, we hope our efforts will shed light on other edge-assisted mobile applications.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their insightful comments and Vijay Gopalakrishnan, Eric Zavesky, and Feng Qian for the initial discussion. The research of Bo Han was funded in part by 4-VA, a collaborative partnership for advancing the Commonwealth of Virginia. The research of Pan Hui was supported in part by the General Research Fund from the Research Grants Council of Hong Kong under Grant 26211515 and Grant 16214817 and the 5GEAR project (Decision No. 318927) and the FIT project (Decision No. 325570) funded by the Academy of Finland.

REFERENCES

- [1] CMU-Oxford Sculpture Dataset. <http://www.robots.ox.ac.uk/~vgg/data/sculptures/>. [accessed on 22-January-2022].
- [2] ImageNet. <http://image-net.org/>. [accessed on 22-January-2022].
- [3] The Paintings Dataset. <http://www.robots.ox.ac.uk/~vgg/data/paintings/>. [accessed on 22-January-2022].
- [4] K. Ahuja, M. Goel, and C. Harrison. BodySLAM: Opportunistic User Digitization in Multi-User AR/VR Experiences. In *Proceedings of Symposium on Spatial User Interaction (SUI)*, 2020.
- [5] A. Alahi, R. Ortiz, and P. Vanderghenst. Freck: Fast Retina Keypoint. In *Proceedings of IEEE/CVF CVPR*, 2012.
- [6] G. Ananthanarayanan, P. Bahl, P. Bodík, K. Chintalapudi, M. Philipose, L. R. Sivalingam, and S. Sinha. Real-Time Video Analytics: The Killer App for Edge Computing. *IEEE Computer*, 50(10):58–67, 2017.
- [7] K. Apicharttrisor, X. Ran, J. Chen, S. V. Krishnamurthy, and A. K. Roy-Chowdhury. Frugal Following: Power Thrifty Object Detection and Tracking for Mobile Augmented Reality. In *Proceedings of ACM SenSys*, 2019.
- [8] Apple Inc. Creating a Multiuser AR Experience. <https://apple.co/2tG5DrQ>, 2018. [accessed on 22-January-2022].
- [9] H. Bay, T. Tuytelaars, and L. Van Gool. SURF: Speeded Up Robust Features. In *Proceedings of European Conference on Computer Vision (ECCV)*, 2006.
- [10] J. T. Bieh, M. Cooper, G. Filby, and S. Kratz. LoCo: a ready-to-deploy framework for efficient room localization using Wi-Fi. In *Proceedings of ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp)*, 2014.
- [11] M. Björkman, N. Bergström, and D. Kragic. Detecting, segmenting and tracking unknown objects using multi-label MRF inference. *Computer Vision and Image Understanding*, 118:111–127, 2014.
- [12] L. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker. Web Caching and Zipf-like Distributions: Evidence and Implications. In *Proceedings of IEEE International Conference on Computer Communications (INFOCOM)*, 1999.
- [13] A. Butz, T. Höllerer, S. Feiner, B. MacIntyre, and C. Beshers. Enveloping Users and Computers in a Collaborative 3D Augmented Reality. In *Proceedings of IEEE and ACM International Workshop on Augmented Reality*, 1999.
- [14] K. Chen, T. Li, H.-S. Kim, D. E. Culler, and R. H. Katz. MARVEL: Enabling Mobile Augmented Reality with Low Energy and Low Latency. In *Proceedings of ACM Conference on Embedded Networked Sensor Systems (SenSys)*, 2018.
- [15] T. Y.-H. Chen, L. Ravindranath, S. Deng, P. Bahl, and H. Balakrishnan. Glimpse: Continuous, Real-Time Object Recognition on Mobile Devices. In *Proceedings of ACM Conference on Embedded Networked Sensor Systems (SenSys)*, 2015.
- [16] Y. Cheng. Edge caching and computing in 5G for mobile augmented reality and haptic internet. *Computer Communications*, 158(15):24–31, 2020.
- [17] A. Coates, A. Ng, and H. Lee. An Analysis of Single-Layer Networks in Unsupervised Feature Learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics*, 2011.
- [18] E. Cranmer, T. Jung, M. C. tom Dieck, and A. Miller. Understanding the Acceptance of Augmented Reality at an Organisational level: The Case of Geveor Tin Mine Museum. In *Proceedings of International Conference on Information and Communication Technologies in Tourism*, 2016.
- [19] L. Gautier, C. Diot, and J. Kurose. End-to-end transmission control mechanisms for multiparty interactive applications on the Internet. In *Proceedings of IEEE International Conference on Computer Communications (INFOCOM)*, 1999.
- [20] Google. Introduction to Cloud Anchors on Android. <https://developers.google.com/ar/develop/java/cloud-anchors/overview-android>, 2018. [accessed on 22-January-2022].
- [21] B. Han, P. Hui, V. A. Kumar, M. Marathe, J. Shao, and A. Srinivasan. Mobile Data Offloading through Opportunistic Communications and Social Participation. *IEEE Transactions on Mobile Computing*, 11(5):821–834, 2012.
- [22] S. Han, H. Shen, M. Philipose, S. Agarwal, A. Wolman, and A. Krishnamurthy. MCDNN: An Approximation-Based Execution Framework for Deep Stream Processing Under Resource Constraints. In *Proceedings of ACM MobiSys*, 2016.
- [23] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. In *Proceedings of IEEE/CVF CVPR*, 2016.
- [24] A. Henrysson, M. Billinghurst, and M. Ollila. Face to Face Collaborative AR on Mobile Phones. In *Proceedings of IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR)*, 2005.
- [25] B. K. Horn and B. G. Schunck. Determining Optical Flow. *Artificial Intelligence*, 17(1-3):185–203, 1981.
- [26] S. Ioannidis, L. Massoulié, and A. Chaintreau. Distributed Caching over Heterogeneous Mobile Networks. In *Proceedings of ACM SIGMETRICS*, 2010.
- [27] P. Jain, J. Manweiler, and R. Roy Choudhury. OverLay: Practical Mobile Augmented Reality. In *Proceedings of ACM MobiSys*, 2015.
- [28] P. Jain, J. Manweiler, and R. Roy Choudhury. Low Bandwidth Offload for Mobile AR. In *Proceedings of ACM CoNEXT*, 2016.
- [29] S. Jana, D. Molnar, A. Moshchuk, A. Dunn, B. Livshits, H. J. Wang, and E. Ofek. Enabling Fine-Grained Permissions for Augmented Reality Applications with Recognizers. In *Proceedings of USENIX Security Symposium*, 2013.
- [30] A. Juan and E. Vidal. Bernoulli mixture models for binary images. In *Proceedings of the 17th International Conference on Pattern Recognition (ICPR)*, 2004.
- [31] L. Juan and O. Gwun. A Comparison of SIFT, PCA-SIFT and SURF. *International Journal of Image Processing*, 3(4):143–152, 2009.
- [32] H. Kato, M. Billinghurst, I. Poupyrev, K. Imamoto, and K. Tachibana. Virtual Object Manipulation on a Table-Top AR Environment. In *Proceedings of IEEE and ACM International Symposium on Augmented Reality*, 2000.
- [33] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet Classification with Deep Convolutional Neural Networks. In *Proceedings of NIPS*, 2012.
- [34] K. Lebeck, T. Kohno, and F. Roesner. How to Safely Augment Reality: Challenges and Directions. In *Proceedings of ACM HotMobile*, 2016.
- [35] K. Lebeck, K. Ruth, T. Kohno, and F. Roesner. Securing Augmented Reality Output. In *Proceedings of IEEE Symposium on Security and Privacy*, 2017.
- [36] S. Leutenegger, M. Chli, and R. Y. Siegwart. BRISK: Binary Robust Invariant Scalable Keypoints. In *Proceedings of ICCV*, 2011.
- [37] K. C.-J. Lin, C.-W. Chen, and C.-F. Chou. Preference-aware content dissemination in opportunistic mobile social networks. In *Proceedings of IEEE INFOCOM*, 2012.
- [38] L. Liu, H. Li, and M. Gruteser. Edge Assisted Real-time Object Detection for Mobile Augmented Reality. In *Proceedings of ACM MobiCom*, 2019.
- [39] Z. Liu, G. Lan, J. Stojkovic, Y. Zhang, C. Joe-Wong, and M. Gorlatova. CollabAR: Edge-assisted Collaborative Image Recognition for Mobile Augmented Reality. In *Proceedings of ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*, 2020.
- [40] M. A. Livingston, L. J. Rosenblum, D. G. Brown, G. S. Schmidt, S. J. Julier, Y. Baillot, J. E. S. II, Z. Ai, and P. Maassel. Military Applications of Augmented Reality. In B. Furht, editor, *Handbook of Augmented Reality*, pages 671–706. Springer, 2011.
- [41] D. G. Lowe. Distinctive image features from scale-invariant keypoints. *Journal of Computer Vision*, 60(2):91–110, 2004.
- [42] W. Ma, L. Cao, L. Yu, G. Long, and Y. Li. GPU-FV: Realtime Fisher Vector and Its Applications in Video Monitoring. In *Proceedings of the ACM International Conference on Multimedia Retrieval (ICMR)*, 2016.
- [43] S. Maheshwari, D. Raychaudhuri, I. Seskar, and F. Bronzino. Scalability and Performance Evaluation of Edge Cloud Systems for Latency Constrained Applications. In *Proceedings of ACM/IEEE SEC*, 2018.
- [44] L. McNamara, C. Mascolo, and L. Capra. Media Sharing based on Colocation Prediction in Urban Transport. In *Proceedings of ACM Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2008.
- [45] NVIDIA. CUDA. <https://developer.nvidia.com/about-cuda>, 2017. [accessed on 22-January-2022].
- [46] T. Ohshima, K. Satoh, H. Yamamoto, and H. Tamura. AR² Hockey: A Case Study of Collaborative Augmented Reality. In *Proceedings of the Virtual Reality Annual International Symposium*, 1998.
- [47] OpenCV. OpenCV. <http://opencv.org/>, 2017. [accessed on 22-January-2022].
- [48] OpenCV. OpenCV4Android. <https://opencv.org/android/>, 2017. [accessed on 22-January-2022].
- [49] F. Perronnin, Y. Liu, J. Sánchez, and H. Poirier. Large-scale image retrieval with compressed Fisher vectors. In *Proceedings of IEEE/CVF CVPR*, 2010.
- [50] F. Perronnin, J. Sánchez, and T. Mensink. Improving the Fisher Kernel for

- Large-Scale Image Classification. In *Proceedings of ECCV*, 2010.
- [51] D. N. E. Phon, M. B. Ali, and N. D. A. Halim. Collaborative Augmented Reality in Education: A Review. In *Proceedings of International Conference on Teaching and Learning in Computing and Engineering*, 2014.
 - [52] X. Ran, H. Chen, X. Zhu, Z. Liu, and J. Chen. DeepDecision: A Mobile Deep Learning Framework for Edge Video Analytics. In *Proceedings of INFOCOM*, 2018.
 - [53] X. Ran, C. Slocum, M. Gorlatova, and J. Chen. ShareAR: Communication-Efficient Multi-User Mobile Augmented Reality. In *Proceedings of ACM HotNets*, 2019.
 - [54] X. Ran, C. Slocum, Y.-Z. Tsai, K. Apicharttrisorn, M. Gorlatova, and J. Chen. Multi-User Augmented Reality with Communication Efficient and Spatially Consistent Virtual Objects. In *Proceedings of ACM CoNEXT*, 2020.
 - [55] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You Only Look Once: Unified, Real-Time Object Detection. In *Proceedings of IEEE/CVF CVPR*, 2016.
 - [56] H. T. Regenbrecht, M. T. Wagner, and G. Barattoff. MagicMeeting: A Collaborative Tangible Augmented Reality System. *Virtual Reality*, 6(3):151–166, 2002.
 - [57] G. Reitmayr and D. Schmalstieg. Mobile Collaborative Augmented Reality. In *Proceedings of IEEE and ACM International Symposium on Augmented Reality (ISAR)*, 2001.
 - [58] P. Ren, X. Qiao, Y. Huang, L. Liu, C. Pu, S. Dustdar, and J.-L. Chen. Edge AR X5: An Edge-Assisted Multi-User Collaborative Framework for Mobile Web Augmented Reality in 5G and Beyond. *IEEE Transactions on Cloud Computing (Early Access)*, pages 1–17, 2021.
 - [59] D. A. Reynolds, T. F. Quatieri, and R. B. Dunn. Speaker Verification Using Adapted Gaussian Mixture Models. *Digital Signal Processing*, 10(1-3):19–41, 2000.
 - [60] F. Roesner, T. Kohno, and D. Molnar. Security and Privacy for Augmented Reality Systems. *Communications of the ACM*, 57(4):88–96, 2014.
 - [61] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski. ORB: An efficient alternative to SIFT or SURF. In *Proceedings ICCV*, 2011.
 - [62] Y.-J. Seo, J. Lee, J. Hwang, D. Niyato, H.-S. Park, and J. K. Choi. A Novel Joint Mobile Cache and Power Management Scheme for Energy-Efficient Mobile Augmented Reality Service in Mobile Edge Computing. *IEEE Wireless Communications Letters*, 10(5):1061–1065, 2021.
 - [63] V. Srinivasan, M. Motani, and W. T. Ooi. Analysis and Implications of Student Contact Patterns Derived from Campus Schedules. In *Proceedings of ACM Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2006.
 - [64] Z. Szalavári, D. Schmalstieg, A. L. Fuhrmann, and M. Gervautz. “Studierstube”: An environment for collaboration in augmented reality. *Virtual Reality*, 3(1):37–48, 1998.
 - [65] VLFeat. VLFeat. <http://www.vlfeat.org/>, 2017. [accessed on 22-January-2022].
 - [66] D. Wagner, T. Pintaric, F. Ledermann, and D. Schmalstieg. Towards Massively Multi-user Augmented Reality on Handheld Devices. In *Proceedings of International Conference on Pervasive Computing (Pervasive)*, 2005.
 - [67] J. Wang, Z. Feng, S. George, R. Iyengar, P. Pillai, and M. Satyanarayanan. Towards scalable edge-native applications. In *Proceedings of ACM/IEEE Symposium on Edge Computing (SEC)*, 2019.
 - [68] A. Younis, B. Qiu, and D. Pompili. Latency-aware Hybrid Edge Cloud Framework for Mobile Augmented Reality Applications. In *Proceedings of IEEE International Conference on Sensing, Communication, and Networking (SECON)*, 2020.
 - [69] C. S. Yusof, T. S. Low, A. W. Ismail, and M. S. Sunar. Collaborative Augmented Reality for Chess Game in Handheld Devices. In *Proceedings of IEEE Conference on Graphics and Media (GAME)*, 2019.
 - [70] W. Zhang, B. Han, and P. Hui. Jaguar: Low Latency Mobile Augmented Reality with Flexible Tracking. In *Proceedings of ACM Multimedia*, 2018.
 - [71] W. Zhang, B. Han, P. Hui, V. Gopalakrishnan, E. Zavesky, and F. Qian. CARS: Collaborative Augmented Reality for Socialization. In *Proceedings of International Workshop on Mobile Computing Systems and Applications (HotMobile)*, 2018.
 - [72] F. Zhou, H. B.-L. Duh, and M. Billinghurst. Trends in augmented reality tracking, interaction and display: A review of ten years of ISMAR. In *Proceedings of IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR)*, 2008.